

1.Optimización en sistemas financieros:

a. Imagina que trabajas en una aplicación bancaria que debe procesar transacciones en tiempo real. ¿Qué estrategias aplicarías para mejorar el rendimiento y la escalabilidad? Considera aspectos como concurrencia, caché y patrones de diseño

R/= Principalmente para mantener un buen rendimiento en caso de muchas transacciones concurrentes manejaría Spring WebFlux para que en los tiempo de espera de proceso I/O no se suspenda el hilo y se pueda seguir trabajando sobre otros procesos agilizando el sistema. Además como se puede manejar concurrencia sobre una misma cuenta esto generaría problemas en la información de la BD. Por lo tanto, yo aplicaría restricciones de concurrencia utilizando SELECT ... FOR UPDATE. De esta manera evitando que se modifique en concurrencia un mismo registro y pierda veracidad la información. También depende de la arquitectura y de si el despliegue lo permite manejaría un sistema de colas como podría ser Kafka o RabbitMQ.

2. Seguridad en APIs financieras:

a. Explica cómo protegerías una API que maneja información sensible de cuentas bancarias contra ataques como inyección SQL, CSRF, XSS y otros ataques comunes

R/= Para evitar inyecciones SQL al momento de crear consultas con parámetros, no usaría un replace o modificaría directamente la consulta. En vez de eso utilizaría parámetros denotados como por ejemplo :idUsuario para que no afecte a la consulta SQL. Para evitar casos de CSRF usaría tokens de la estructura Authorization: Bearer en vez de simplemente cookies o tokens anti-CSRF. Para XSS mantendría los consumos y respuestas en el formato JSON para evitar que se realicen ejecuciones maliciosas.

3. Transacciones en sistemas distribuidos:

a. En un sistema bancario distribuido, ¿cómo implementarías la consistencia y el manejo de errores en una API que procesa transferencias entre cuentas en diferentes servicios?

R/= Principalmente manejaría transacciones en bloque para evitar el caso común de realizar un pago y que descuente el dinero pero no se almacene por un error en esta segunda operación. Por lo que haría un plan y manejaría los métodos con `@Transactional` indicándole un rollback en caso de que alguno de los métodos agrupados falle. (A menos que sea un error controlado por la aplicación)

4. Pruebas unitarias y de integración:

- a. Describe cómo diseñarías una suite de pruebas que asegure la correcta operación de una API bancaria, considerando tanto pruebas unitarias como de integración. ¿Qué herramientas utilizarías?

R/= Principalmente se distribuiría de pruebas unitarias en el backend usando Junit y Mockito para garantizar el buen funcionamiento de la lógica de negocio. Luego en el frontend usando Jest para las pruebas unitarias del frontend, logrando de esta manera probar los flujos de ambos de manera aislada para identificar mejor si es un error de lógica o de integraciones. Y para pruebas de integración se pueden usar Testcontainers para levantar de manera temporal una BD para las pruebas.

5. Front-end:

- a. En una aplicación bancaria que muestra el saldo de las cuentas, ¿cómo gestionarías el estado y la autenticación en el front-end, garantizando la seguridad y la coherencia de los datos?

R/= Utilizaría un método de 2 step verification, para evitar inicios de sesión maliciosos. Se puede manejar tanto con una clave dinámica, como con un SMS al momento de ingresar y de realizar transacciones o movimientos sensibles.

SPRING BOOT:

- a. Que pasa en una aplicación internamente cuando usas la anotación `@SpringBootApplication` y cómo afecta el arranque de una aplicación?

R/= Inicia creando la aplicación de Spring, luego toma las configuraciones anotadas en el `application.properties`, ya teniendo esta información/configuración lo que hace es crear el contexto de la aplicación. Con este contexto creado empiezan a cargar los Bean, con sus debidas anotaciones e inyecciones de dependencia. Si todo se registra

correctamente y no hay errores , procede a ejecutar la aplicación y publicarla en el puerto asignado o por defecto en el 8080.

- b. ¿Cómo funciona el ciclo de vida de un beans en Spring y como podrías intervenir en él?

R/= El Bean tiene varias etapas que se basan en el siguiente proceso: Se definen, se instancian, luego se le inyectan sus dependencias, luego se inicializan y ya estarían listos para usarse, hasta que se destruyen en el cierre de la aplicación.

Se puede intervenir en el ciclo de vida usando estas 2 anotaciones:

@PostConstruct

@PreDestroy

Que sirven para ejecutar ciertas acciones después de construirse y antes de destruirse. Es como OnInit y OnDestroy de los componentes de Angular

- c. ¿Cómo personalizarías el comportamiento de la auto-configuración en Spring Boot sin romper la filosofía de 'convención sobre configuración'?

R/= La más común y menos invasiva es modificando los parámetros en el application.properties y con eso debería bastar para el 90% de las situaciones que se quieren personalizar en el arranque de Spring Boot. En dado caso de requerir algo más técnico se puede definir una configuración de AutoArranque personalizada en una clase de Java, pero es mas invasivo que la opción anterior.